

Relative path: README.md

Filename: README.md Extension: .md

OASIS – Operational Anomaly & Security Impact System

AI-Powered Behavioral Anomaly Detection for Cybersecurity – an OT/ICS instance of the official problem statement, built end to end: synthetic access-log generation, baseline behavioral profiling, sequence-aware detection, anomaly-type classification, explainable risk scoring, grounded LLM reasoning, and an analyst dashboard.

> Traditional UEBA stops at detection. OASIS translates behavioral anomalies into
> operational risk – combining behavior modeling, asset criticality, MITRE ATT&CK
> mapping, and grounded, explainable analyst guidance.

Full write-up: [``OASIS_Report.docx``](./OASIS_Report.docx) – read this for methodology, metrics, and known limitations. This README is the quick-start and orientation layer.

Headline results (final verified run)

```
| Metric | Value |
|---|---|
| Sessions scored | 1,333 (103 labeled attack, 7.7%) |
| Precision @ top 1% alert budget | **76.9%** (the brief's own "realistic analyst alert budget" metric) |
| Detection F1 (best threshold, 0.55) | 0.5053 |
| Per-class classification accuracy (true positives) | 48/48 correct across all 7 attack types |
| Groundedness (LLM narratives) | 87/87 passed |
| Concept drift | `insider_drift` entity confirmed **not** permanently flagged (0.1903 → 0.1783 mean score) |
| Runtime | 7.91s, single pass, single machine, no GPU |
```

Full metrics, per-class tables, and the false-positive misrouting breakdown are in the report, Section 8.

Architecture

...

```
[Synthetic Generator]      data_gen/
    ■ Session (typed, no label field)
    ▼
[Layer 0: Baseline Profiling] profiling/
    EWMA + cold-start prior + drift-snapshot
    ■ rarity_score, geo_velocity_flag, drift_score, is_cold_start
    ▼
[Layer 1: Sequence Detection] detection/
    GRU autoencoder (from-scratch NumPy) + cold-start rule engine
    ■ AnomalyEvent (anomaly_score, contributing_features)
    ▼
[Layer 2: Anomaly Classification] classification/
    RandomForest, flagged-sessions-only, with an explicit "uncertain"
    confidence floor instead of forcing a guess
    ■ ClassifiedAnomaly (anomaly_type, confidence)
    ▼
[Layer 3: Risk Scoring]      scoring/
    Transparent arithmetic – no model where one isn't needed
    ■ RiskAssessment (impact_score, recommended_action)
    ▼
[Layer 4: Reasoning]         reasoning/
    Grounded LLM narrative + automated groundedness check
    ■ IncidentNarrative
    ▼
[Layer 5: Analyst Dashboard] api/ + dashboard/
    Ranked alert queue, entity history, contributing factors
    ...
```

Every arrow above is a typed Pydantic contract (``contracts/schemas.py``), imported by every layer – no layer can silently drift a field name.

Repo structure

...

```

oasis/
■■■ data_gen/          synthetic log + attack-taxonomy generator
■■■ profiling/         Layer 0 – baseline profiler, drift snapshot, geo-velocity check
■■■ detection/         Layer 1 – GRU autoencoder + signal combination
■■■ classification/    Layer 2 – RandomForest + confidence floor
■■■ scoring/           Layer 3 – risk engine + MITRE map + asset criticality
■■■ reasoning/         Layer 4 – grounded narrative generation
■■■ api/ + dashboard/  Layer 5 – FastAPI backend + analyst UI
■■■ eval/              metrics.py – precision/recall/F1, precision@top-1%,
■■                      cold-start eval, concept-drift eval, misrouting diagnostic
■■■ contracts/         schemas.py – every typed contract, single source of truth
■■■ run_pipeline.py     runs Layers 0-3 end to end, threshold sweep, writes results/
■■■ run_reasoning.py    runs Layer 4 over Layer 3's output
■■■ run_streaming_demo.py lightweight streaming-feasibility demo (see report §9)
■■■ results/           generated: metrics.json, risk_assessments.json,
                        incident_narratives.json, worked_example_drift_profile.json
...

```

Quick start

```

```bash
pip install -r requirements.txt
1. Generate synthetic data (deterministic – seeded)
python data_gen/generate_logs.py

2. Run Layers 0-3 (training, detection, classification, risk scoring)
python run_pipeline.py

3. Run Layer 4 (grounded incident narratives)
python run_reasoning.py

4. (Optional) Run the streaming-feasibility demo – replays sessions.jsonl
one at a time in timestamp order, scoring each on arrival rather than
in a single batch pass. Produces results/streaming_demo_log.txt.
python run_streaming_demo.py

5. Start the dashboard
uvicorn api.main:app --reload
open http://localhost:8000
```

```

Result: ****87 alerts raised**** – identical to the batch pipeline's 48 TP + 39 FP – confirming session-by-session streaming replay produces the same result as batch scoring. The replay uses an independently but identically fitted detector (same seeded training process, not a reused model file), which is itself a stronger claim: the whole pipeline is fully deterministic end to end, not just that one trained model was reused twice. Mean per-session scoring latency: ****0.95ms**** (max 4.2ms – ordinary timing variance, not correlated with session type or anomaly score).

This is not a production message-queue deployment – see the report, Section 9, for what a real deployment would add (Kafka/MQTT ingestion, an external per-entity state store for multi-worker scoring).

All randomness (data generation, GRU init, classifier) is seeded (``RNG_SEED = 42``) – reruns without a code change reproduce the exact same numbers.

What makes this different from "we built UEBA with an LLM"

1. ****Detection and classification are separate models on purpose**** – detection is unsupervised (trained only on normal sessions) so it never fights class imbalance; classification only ever runs on sessions already flagged.
2. ****Cold-start and concept drift are measured, not just claimed**** – real before/after numbers, not a bullet point.
3. ****A dedicated, non-averaged geo-velocity signal**** – added after discovering that averaging it into a 9-feature rarity score silently diluted it to invisibility.
4. ****The classifier can say "uncertain" instead of being forced to guess** – a deliberate fix after finding that a forced guess actively misdirects a SOC analyst more than an honest "this doesn't clearly match a known pattern."
5. ****A grounded LLM layer with a live, verifiable groundedness check**** – not "trust the model."

Three real bugs were found, root-caused, fixed, and re-verified during this project's own internal review – see the report, Section 10.2, for the full diagnostic story on

each.

Known limitations (see report, Section 10, for the full breakdown)

- Batch/static pipeline – designed to be streaming-compatible, not deployed as such.
- Trained on synthetic sessions only; no real-world generalization claim.
- Risk-scoring weights and thresholds are principled starting values, not tuned against real incident data.
- `privilege\_escalation` and `credential\_stuffing` have thin detection-layer recall relative to their injection volume – an open item, named honestly rather than hidden.

Team

Built for the AI-Powered Behavioral Anomaly Detection for Cybersecurity problem statement. See `OASIS\_Report.docx` for the full methodology and metrics writeup, and the accompanying slide deck for the condensed pitch version.